BRENT OZAR
UNLIMITED®

## First, a Reminder...

Deck 4 p1

## This class is about faster AND cheaper.

I'm about to discuss hardware fixes for storage bottlenecks.

These fixes will cost money.

If you don't have PAGEIOLATCH as your top wait, don't use these techniques and fixes.

Save your money! Focus on YOUR bottleneck.

Deck 4 p2

© Brent Ozar Unlimited. All rights reserved.

BRENT OZAR
UNLIMITED®

# Cloud Storage is a 2010s Throwback

Deck 4 p3

## Let's take this AWS RDS instance:

```
sp_BlitzFirst @SinceStartup = 1
```

150 % No issues found

Results | Messages

| | Pattern | Hours Sample | Thread Time (Hours) | wait_type | wait_category | Wait Time (Hours) | Avg ms Per Wait |
|---|---|---|---|---|---|---|---|
| 1 | WAIT STATS | 218.0 | 2493.7 | PAGEIOLATCH_SH | Buffer IO | 1485.1 | 52.2 |
| 2 | WAIT STATS | 218.0 | 2493.7 | CXCONSUMER | Parallelism | 748.2 | 305.9 |
| 3 | WAIT STATS | 218.0 | 2493.7 | CXSYNC_PORT | Other | 214.7 | 52656.7 |
| 4 | WAIT STATS | 218.0 | 2493.7 | ASYNC_NETWORK_IO | Network IO | 15.4 | 137.5 |
| 5 | WAIT STATS | 218.0 | 2493.7 | LCK_M_S | Lock | 9.6 | 264472.7 |
| 6 | WAIT STATS | 218.0 | 2493.7 | SOS_SCHEDULER_YIELD | CPU | 6.3 | 1.7 |
| 7 | WAIT STATS | 218.0 | 2493.7 | CXPACKET | Parallelism | 3.5 | 4.7 |

Deck 4 p4

© Brent Ozar Unlimited. All rights reserved.

## Agenda

Where hardware shows up in query plans

Hardware evolution from 2000s to 2020s to cloud

Why cloud storage performance is catastrophic

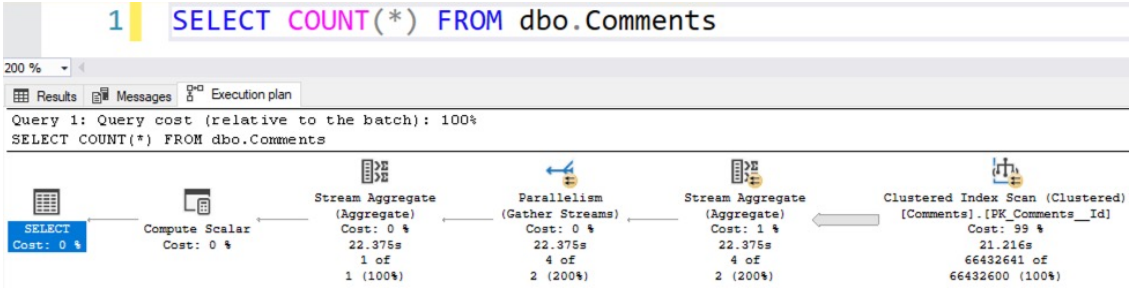
Why memory isn't the answer it was on-premises

Dealing with 2000s-quality hardware in the cloud

- Groups of drives to pool performance
- Choosing your VM type to maximize throughput

© Brent Ozar Unlimited. All rights reserved.

# Execution plans stop at indexes.




But there's more to the right.

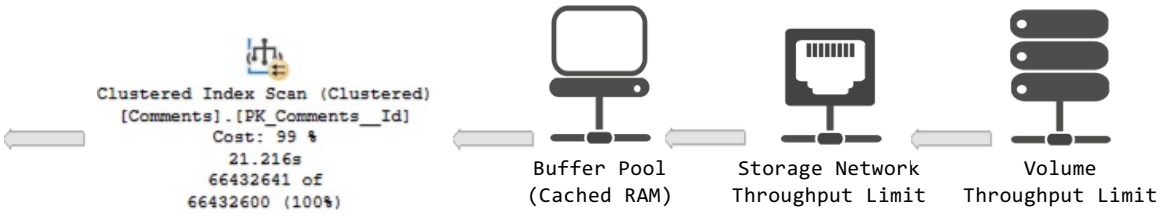
Deck 4 p7

# How does this part really work?

Deck 4 p8

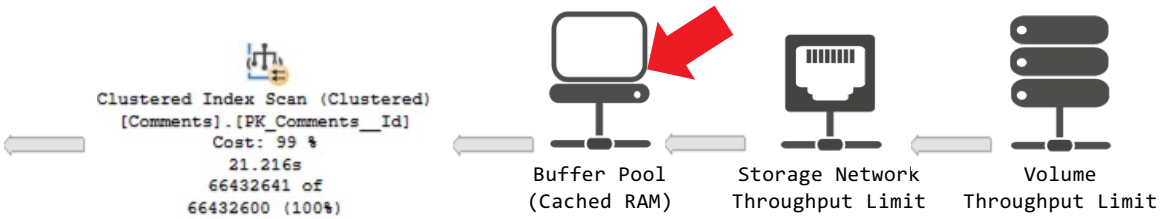
© Brent Ozar Unlimited. All rights reserved.

# The plan really keeps going.




Deck 4 p9

# If you can cache all data in RAM



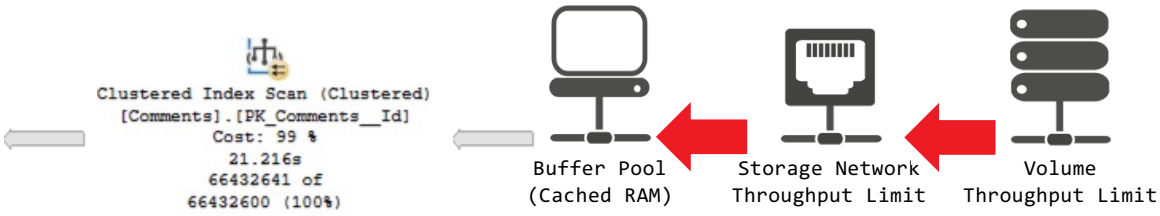

Then storage throughput only matters:

- On startup
- For backups
- For inserts/updates/deletes (to the log file)

Deck 4 p10

© Brent Ozar Unlimited. All rights reserved.

# If you have to read data from disk




Then the smallest red arrow dictates read query performance.

Load testing each arrow was hard.

Deck 4 p11

# This was terrible in the 2000s.

We simply couldn't cache enough data.

We used magnetic hard drives, so:

- Drives only did 150 operations per second (IOPs)
- We pooled drives together in RAID arrays not just for safety, but for speed

1 TB was called a Very Large Database (VLDB)

Deck 4 p12

© Brent Ozar Unlimited. All rights reserved.

## But in the 2010s...

Memory costs dropped from $100/GB to $10/GB:
cache 10x more, cheaply. 128GB = $1,280.

Fusion-io and Intel popularized solid state drives:
IOPs per drive went from 150 to 150,000+

SAN vendors like Pure built fast shared storage:
a shared pool of flash had nearly unlimited throughput

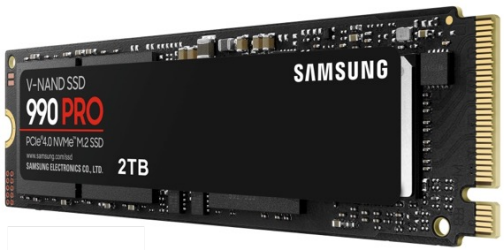
Deck 4 p13

## Amazing time to be alive

Samsung 990 Pro for laptops

Up to about 1.5 million IOPs
(that's 10,000 hard drives)

Up to 7,450 MB/sec
(that's 6 10Gb iSCSI cables)




1TB $100, 2TB $150

Deck 4 p14

© Brent Ozar Unlimited. All rights reserved.

# Better than flying cars

| | 2000s | 2010s | 2020s |
|---|---|---|---|
| Cost for 128GB RAM | $12,800 | $1,280 | $500 |
| Typical drive IOPs | 150 | 150,000 | 1,500,000* |
| Typical drive throughput | 100 MB/sec | 1,000 MB/sec | 7,000 MB/sec* |

* And that Samsung 990 has been out since 2022!

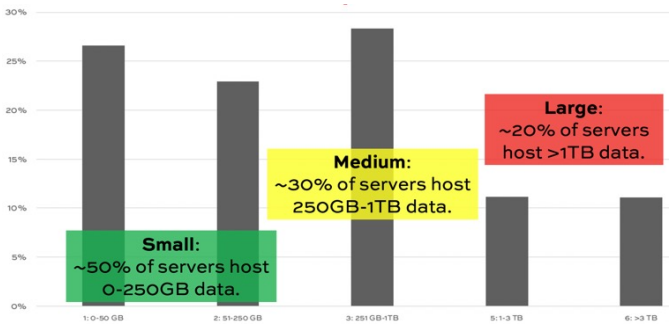
Deck 4 p15

# So we didn't mind growing data.

1 in 5 servers have 1TB or more.

It's not very large now.

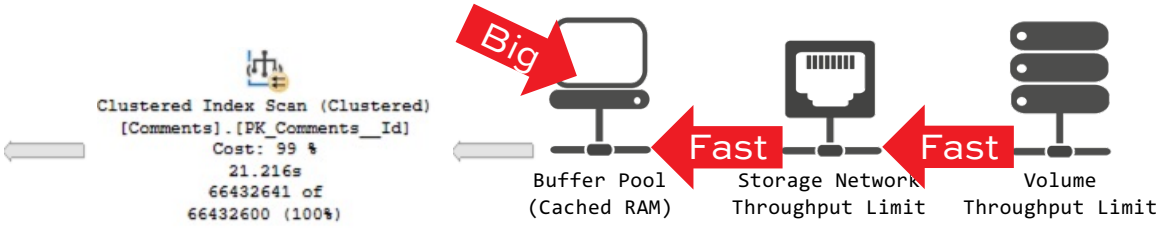
But who cares? Hardware got so good, we could deal with it.




Deck 4 p16

© Brent Ozar Unlimited. All rights reserved.

# In the 2020s

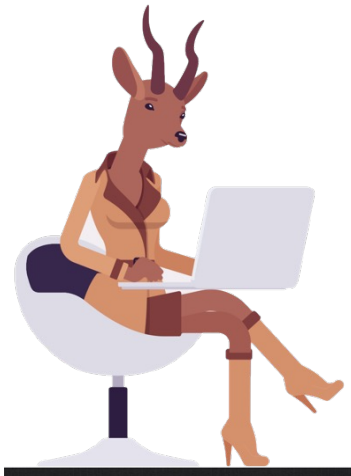



Memory grew so large,
and storage throughput grew so fast,
that many servers just didn't have to worry about it.

Blocking, CPU, parallelism were bigger issues.

Deck 4 p17

# Alex the Architect's hardware



Alex just manages one application & db

The app is growing fast & furious,
users constantly want more features

Can't hire enough good developers

Tends to overprovision hardware,
throwing hardware at problems,
so tends to have high quality hardware

Uses 2020s-grade hardware

Deck 4 p18

© Brent Ozar Unlimited. All rights reserved.

## Billie the Small Biz Bird's hardware

Billie's business sees IT as an expense

The business wants to spend the bare minimum possible to keep apps online

The business will tolerate some outages to save money.

Billie uses a couple/few VM hosts backed by inexpensive shared storage.

Deck 4 p19

## Elliot's enterprise hardware

Elliot's team uses large amounts of hardware from enterprise vendors

A mix of virtualization and bare metal

Refreshes hardware on a scheduled basis, like every 3-5 years

Tries to balance performance and cost

Avoids bleeding edge hardware, so today's hardware is 2010s-grade performance

Deck 4 p20

© Brent Ozar Unlimited. All rights reserved.

## Uh oh

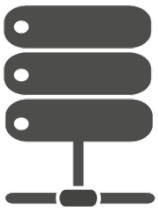
| | 2000s | 2010s | 2020s | 2020s Cloud |
|---|---|---|---|---|
| Cost for 128GB RAM | $12,800 | $1,280 | $500 | $11,700/yr* (E16bds_v5) |
| Typical drive IOPs | 150 | 150,000 | 1,500,000 | 80,000 (Azure Premium SSD v2) |
| Typical drive throughput | 100 MB/sec | 1,000 MB/sec | 7,000 MB/sec | 1,200 MB/sec (Azure Premium SSD v2) |

*This is the cost premium per year for an E32 over an E16, which takes you from 128GB to 256GB. This isn't a perfect measurement – but in the cloud, you can't price components separately.



© Brent Ozar Unlimited. All rights reserved.

## Why so slow & expensive?



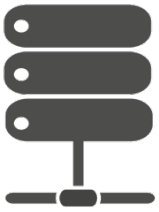
Storage isn't local: it's a shared service.

Storage is a highly reliable building block:

- Individual disks can fail w/o customers knowing
- Servers hosting those disks can also fail
- Customers can't notice lost writes
- Writes must be done on multiple servers, and in most cases, multiple data centers

Deck 4 p23

## Cloud storage prices include:



Multiple drives

In multiple servers

In multiple data centers

Power & networking

Custom software to make it all work

People to build & manage it all

Deck 4 p24

© Brent Ozar Unlimited. All rights reserved.

# And the "fast" cloud storage has SO many gotchas...

- Premium SSD v2 disks can't be used as an OS disk.
- Premium SSD v2 disks can't be used with Azure Compute Gallery.
- Currently, Premium SSD v2 disks are only available in select regions.
- For regions that support availability zones, Premium SSD v2 disks can only be attached to zonal VMs. When creating a new VM or Virtual Machine Scale Set, specify the availability zone you want before adding Premium SSD v2 disks to your configuration.
- Encrypting Premium SSD v2 disks with customer-managed keys using Azure Key Vaults stored in a different Microsoft Entra ID tenant isn't currently supported.
- Azure Disk Encryption (guest VM encryption via BitLocker/DM-Crypt) isn't supported for VMs with Premium SSD v2 disks. We recommend you to use encryption at rest with platform-managed or customer-managed keys, which is supported for Premium SSD v2.
- Currently, Premium SSD v2 disks can't be attached to VMs in Availability Sets.
- Azure Site Recovery isn't supported for VMs with Premium SSD v2 disks.
- Premium SSDv2 doesn't support host caching.

Deck 4

# How does this affect SQL Server?

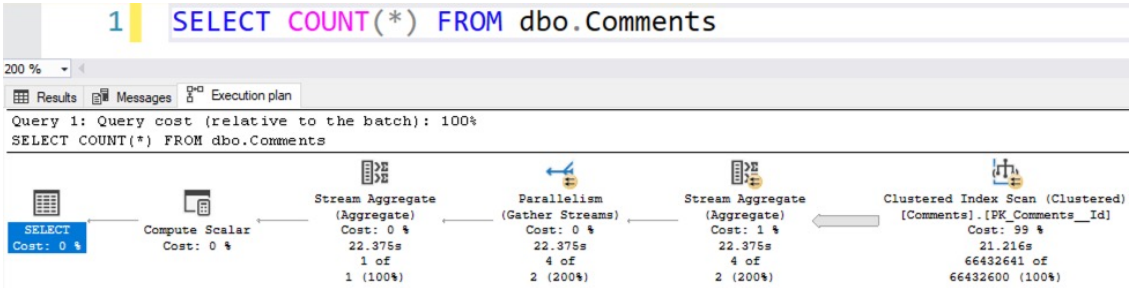
| | 2000s | 2010s | 2020s | 2020s Cloud |
|---|---|---|---|---|
| Cost for 128GB RAM | $12,800 | $1,280 | $500 | $11,700/yr* (E16bds_v5) |
| Typical drive IOPs | 150 | 150,000 | 1,500,000 | 80,000 (Azure Premium SSD v2) |
| Typical drive throughput | 100 MB/sec | 1,000 MB/sec | 7,000 MB/sec | 1,200 MB/sec (Azure Premium SSD v2) |

Deck 4 p26

© Brent Ozar Unlimited. All rights reserved.

# Let's take this query.

My Comments table is just 21GB in size:




Deck 4 p27

# How does this affect SQL Server?

| | 2000s | 2010s | 2020s | 2020s Cloud |
|---|---|---|---|---|
| Cost for 128GB RAM | $12,800 | $1,280 | $500 | $11,700/yr* (E16bds_v5) |
| Typical drive IOPs | 150 | 150,000 | 1,500,000 | 80,000 (Azure Premium SSD v2) |
| Typical drive throughput | 100 MB/sec | 1,000 MB/sec | 7,000 MB/sec | 1,200 MB/sec (Azure Premium SSD v2) |
| Time to read 21GB: | 210 seconds | 21 seconds | 3 seconds | 18 seconds |

Deck 4 p28

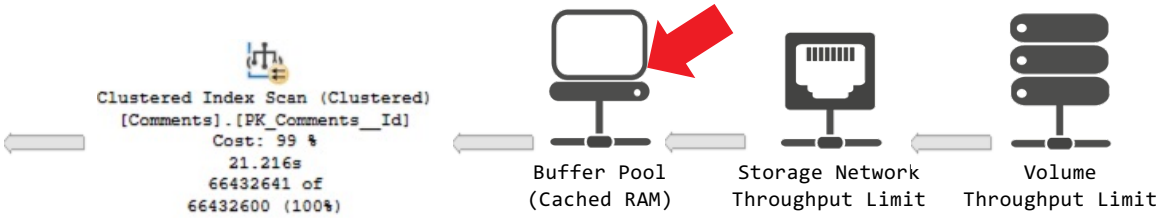
© Brent Ozar Unlimited. All rights reserved.

# Doesn’t memory fix this problem?

Deck 4 p30

© Brent Ozar Unlimited. All rights reserved.

# If you can cache all data in RAM




Then storage throughput only matters:

- On startup
- For backups
- For inserts/updates/deletes (to the log file)

Deck 4 p31

# But if you can't cache it ALL...

Then caching some of a table doesn't usually work.

To keep it simple, say our query is:

```
SELECT *
FROM dbo.Comments
WHERE UserId = 26837     /* Brent */
ORDER BY Id;             /* Clustered primary key */
```

Deck 4 p32

© Brent Ozar Unlimited. All rights reserved.

# The query plan without an index

```
3  SELECT *
4  FROM dbo.Comments
5  WHERE UserId = 26837      /* Brent */
6  ORDER BY Id;              /* Clustered primary key */
```

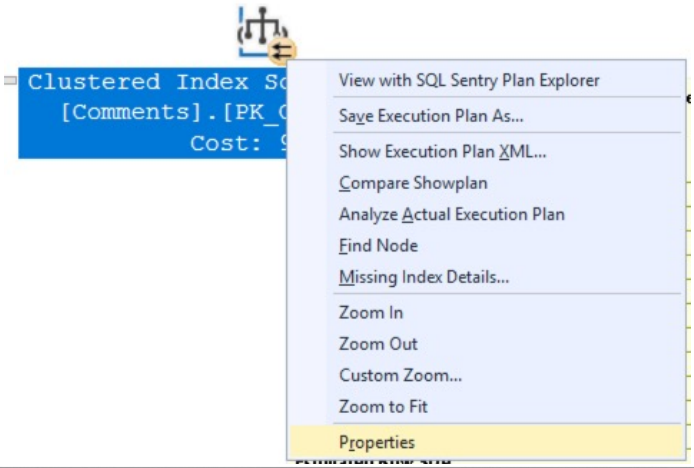
200 %

Messages | Execution plan

```
Query 1: Query cost (relative to the batch): 100%
SELECT * FROM dbo.Comments WHERE UserId = 26837 /* Brent */ ORDER BY Id
Missing Index (Impact 99.9966): CREATE NONCLUSTERED INDEX [<Name of Missing Index, sysname,>] ON [dbo].[Comme
```

Deck 4 p33

# Right-click on the CX scan, click Properties

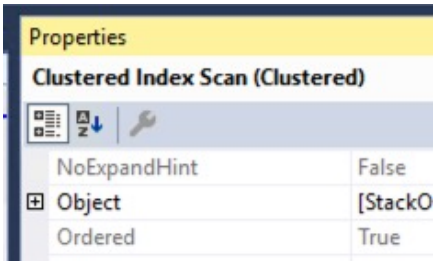



Deck 4 p34

© Brent Ozar Unlimited. All rights reserved.

## Ordered = True

SQL Server is scanning
the clustered index
in order, from start to end




So if you can't cache the entire table,
something terrible happens

Deck 4 p35

## Comments clustered index

The first time the query runs,
SQL Server asks for the first few pages...

Deck 4 p36

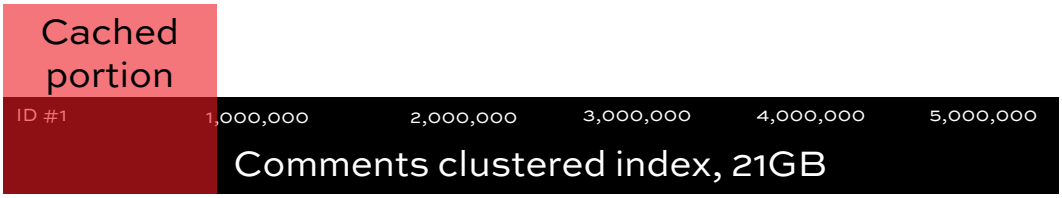
© Brent Ozar Unlimited. All rights reserved.

## Say we have 8GB memory

We may only be able to cache 4GB of the table

The part we're caching is like a sliding window

As our query progresses, the window moves...




Deck 4 p37

## Say we have 8GB memory

We may only be able to cache 4GB of the table

The part we're caching is like a sliding window

As our query progresses, the window moves...

Deck 4 p38

© Brent Ozar Unlimited. All rights reserved.

## Say we have 8GB memory

We may only be able to cache 4GB of the table

The part we're caching is like a sliding window

As our query progresses, the window moves...

Deck 4 p39

## Say we have 8GB memory

We may only be able to cache 4GB of the table

The part we're caching is like a sliding window

As our query progresses, the window moves...

Deck 4 p40

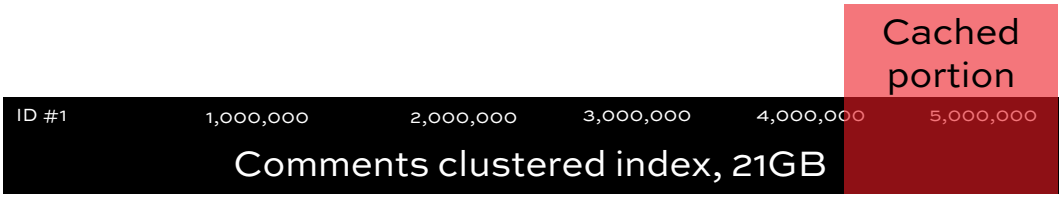
© Brent Ozar Unlimited. All rights reserved.

## When our query finishes

We have some of the Comments table cached...

But...

If the query runs again...




Deck 4 p41

## When the next query starts

SQL Server wants the first few pages

They're not cached, so we have to fetch 'em from disk

Result: constantly hammering storage

Deck 4 p42

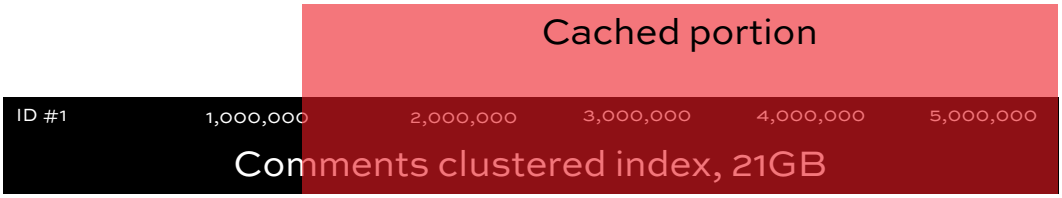
© Brent Ozar Unlimited. All rights reserved.

# Does adding memory help?

Say we upgrade the server to 24GB

And say 18GB of that is available for the buffer pool

When the query finishes, it looks like this:




Deck 4 p43

# We're still screwed.

Adding *some* memory helps when:

- Tables are small
- Readaheads are the problem
- Query workspace is the problem

Caching the entire working data set:
THAT's where the gains are.

More details: https://www.brentozar.com/archive/2022/07/why-adding-some-memory-doesnt-fix-all-caching-problems/

Deck 4 p44

© Brent Ozar Unlimited. All rights reserved.

# How did we cope in the 2000s?

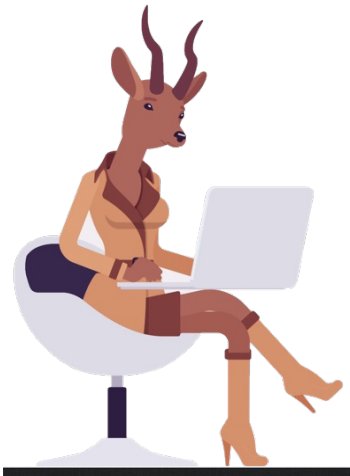
We couldn't cache the entire data set back then: memory was too expensive.

So what did we do back in the 2010s?

| | 2000s | 2010s | 2020s | 2020s Cloud |
|---|---|---|---|---|
| Cost for 128GB RAM | $12,800 | $1,280 | $500 | $11,700/yr* (E16bds_v5) |
| Typical drive IOPs | 150 | 150,000 | 1,500,000 | 80,000 (Azure Premium SSD v2) |
| Typical drive throughput | 100 MB/sec | 1,000 MB/sec | 7,000 MB/sec | 1,200 MB/sec (Azure Premium SSD v2) |

Deck 4 p45

# How Alex the Architect coped



Paid close attention to how servers connected to the storage network

Put a lot of work into picking hardware

Benchmarked HW before going live

Read tech news closely, eager for better options, quickly adopted new stuff like Fusion-IO drives

Deck 4 p46

© Brent Ozar Unlimited. All rights reserved.

## How Billie the Small Biz coped

Tried storage pools: bought more drives, spread the load across ‘em

Teamed 1Gb Ethernet cards together trying to get more throughput

Willing to do one-off server builds with unusual configurations (low CPU, high RAM) to offset problems

Deck 4 p47

## How Elliot’s Enterprise coped

Brought in consultants, HW vendors, MS

Adopted reference configurations like the MS Fast Track Architectures for data warehouses, virtualization

With those standards in place, performance just was what it was: enterprises don’t react fast or react by buying bleeding edge hardware

Deck 4 p48

© Brent Ozar Unlimited. All rights reserved.

## Redundant Array of Independent Drives

Group drives together for:

- Protection: because hard drives failed often, keep multiple copies of data
- Speed: because each one was pretty slow, spread the same data out across lots of drives

Deck 4 p50

© Brent Ozar Unlimited. All rights reserved.

## Redundant Array of Independent Drives

Group drives together for:

- Protection: because hard drives failed often, keep multiple copies of data

Protection matters less in the cloud
not because individual drives are more reliable,
but because failures are mostly invisible to you anyway.
The cloud provider builds in redundancy for that.

Deck 4 p51

## Redundant Array of Independent Drives

Group drives together for:

But this still matters a lot in the 2020s:

- Speed: because each one was pretty slow, spread the same data out across lots of drives

Because cloud storage is pretty crappy.

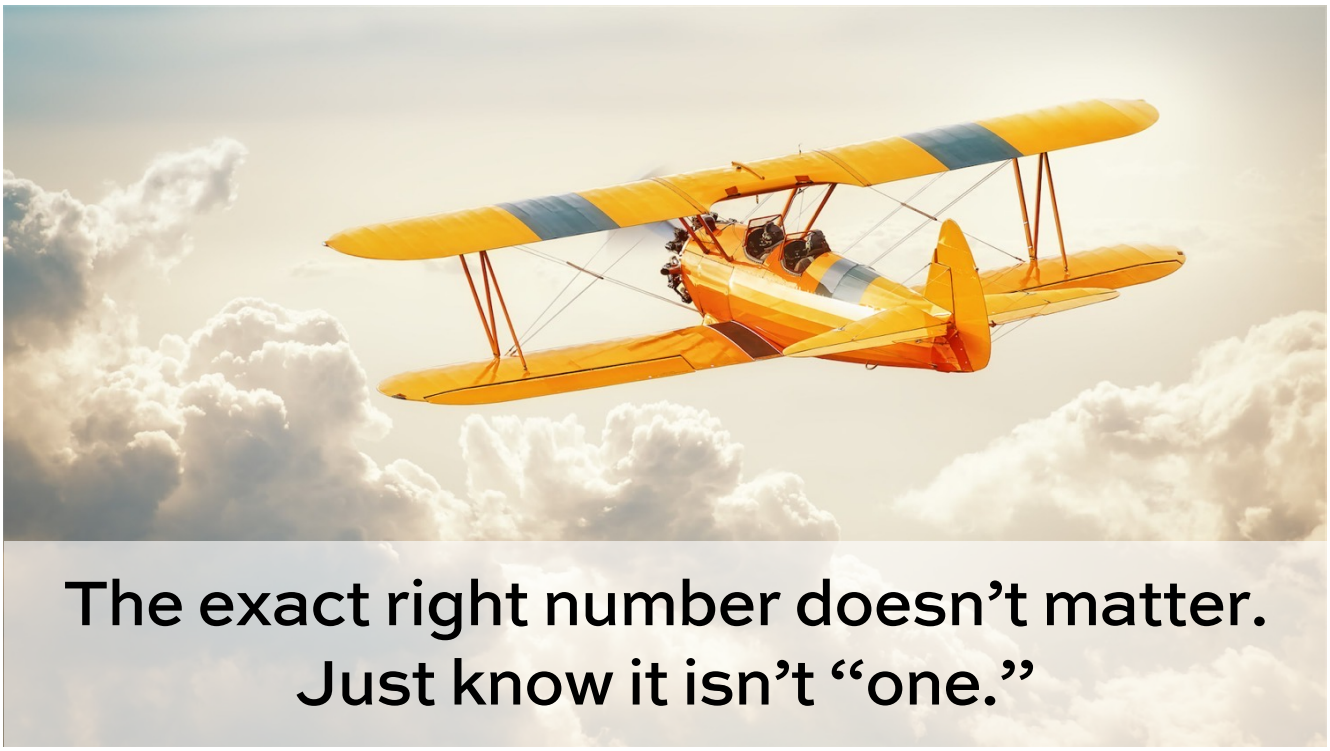
Deck 4 p52

© Brent Ozar Unlimited. All rights reserved.

# How many drives would you need?

| | IOPs (peak burst) | MB/sec | # of drives it would take to match a single laptop SSD |
|---|---|---|---|
| Samsung 990 Pro laptop drive | 1,500,000 | 7,450 | - |
| Azure Premium P30 1TB | 30,000 | 1,000 | 7-50 |
| Azure Premium SSD v2 | 80,000 | 1,200 | 6-19 |
| Azure Ultra | 160,000 | 4,000 | 2-9 |
| AWS GP3 (like Azure Premium) | 16,000 | 1,000 | 7-94 |
| AWS Provisioned IOPs | 64,000 | 1,000 | 8-24 |

Deck 4 p53




© Brent Ozar Unlimited. All rights reserved.

## The old way to use many drives

Present each one as a separate volume to Windows:

- L:\
- M:\
- N:\
- O:\ ... etc

Break each database up into multiple data files

Put each file on its own storage volume

Deck 4 p55

## Problems with that approach

Need to add a new volume for more performance?

- In every database: add a new data file
- Rebuild indexes to spread the data around
- That's a slow, logged activity
- It's really hard to balance things out evenly

Need to subtract a volume because load died off?
Forget it. Quit your job. Move to another city.

Deck 4 p56

© Brent Ozar Unlimited. All rights reserved.

## The new way: let the OS do it

Add another physical volume

Windows admins configure Storage Spaces:
RAID, but managed in software

Windows can even mirror writes across VMs
(but I'm not a fan: remember, we're doing it for speed)

SQL Server just sees one growing logical volume

No need to rebalance data across files*

Deck 4 p57

## * About using multiple data files

More data files doesn't inherently improve reads.

The only way it helps is if those files are spread across multiple volumes, each with their own dedicated throughput.

I do *like* multiple data files per filegroup on 1TB+ databases, for sure.

But once you've already got PAGEIOLATCH waits, and you need to improve performance quickly, adding multiple empty files and rebalancing the data takes weeks.

Deck 4 p58

© Brent Ozar Unlimited. All rights reserved.

## “But what if a drive fails?”

You need alerting just like you’d need on bare metal.

Just like we did back in the 2000s.

If your drive pool has redundancy,
Windows admins need to add another volume in.

If it doesn’t have redundancy (striping),
SQL Server will go down. That’s bad.

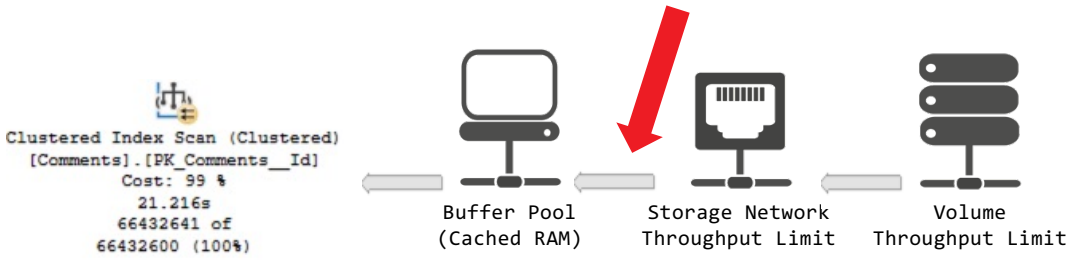
Just like RAID worked back in the 2000s.

Deck 4 p59

## “How fast is fast enough?”

We’ll talk about benchmark testing
in the next module.

But before we get to that,
we need to talk about the other bottleneck.




Deck 4 p60

© Brent Ozar Unlimited. All rights reserved.

© Brent Ozar Unlimited. All rights reserved.

## SQL Server: more, faster drives
## Web servers: boot drive mirror

## That's true in the cloud, too.

Different VM types have different:

- # of CPU cores available
- Makes/models of CPU, different speeds
- Amounts of memory available per CPU core
- Amounts of local ephemeral solid state storage
- Sizes of connectivity to storage

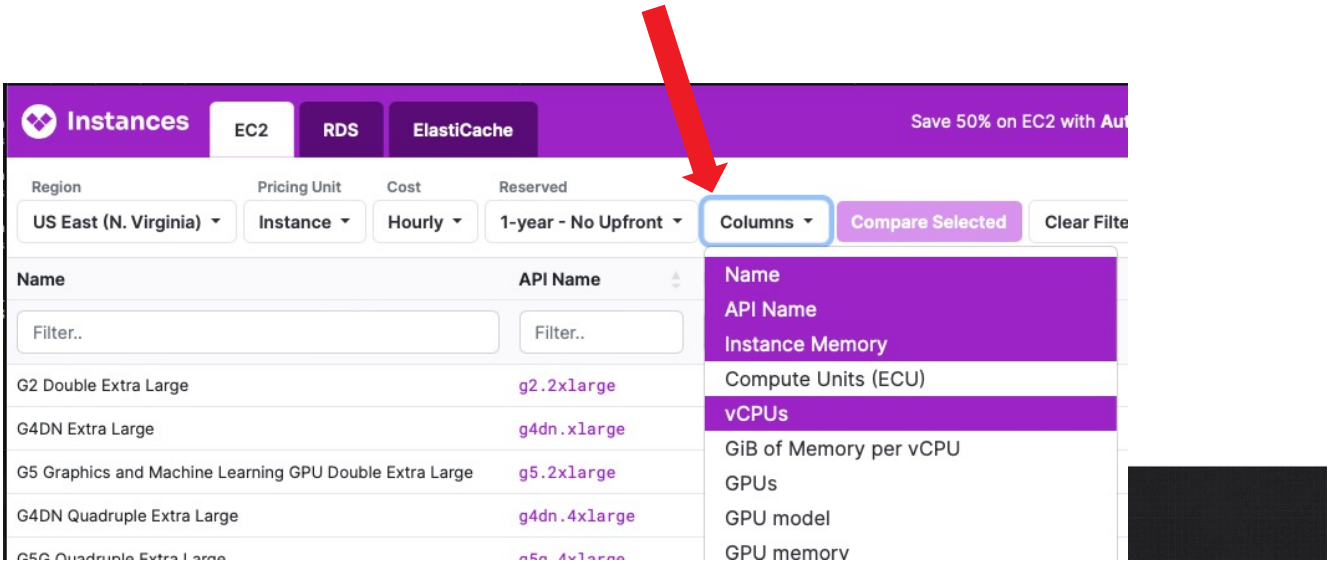
https://instances.vantage.sh/

https://instances.vantage.sh/azure/



© Brent Ozar Unlimited. All rights reserved.

# Click the Columns dropdown, pick the columns you want




# What I like to show

VM columns:

API Name: i3.4xlarge

Instance memory (RAM)

vCPUs (cores)

GiB Memory per vCPU

Clock Speed

Cost per year

AWS storage columns:

EBS Optimized: Max Bandwidth

EBS Optimized: Max Throughput

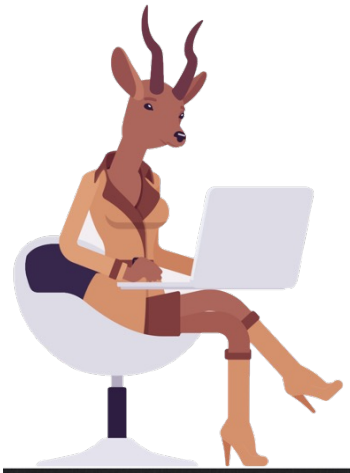
EBS Optimized: Max IOPs

Azure storage columns: Premium Storage ones



© Brent Ozar Unlimited. All rights reserved.

# Let's go through a sizing exercise.

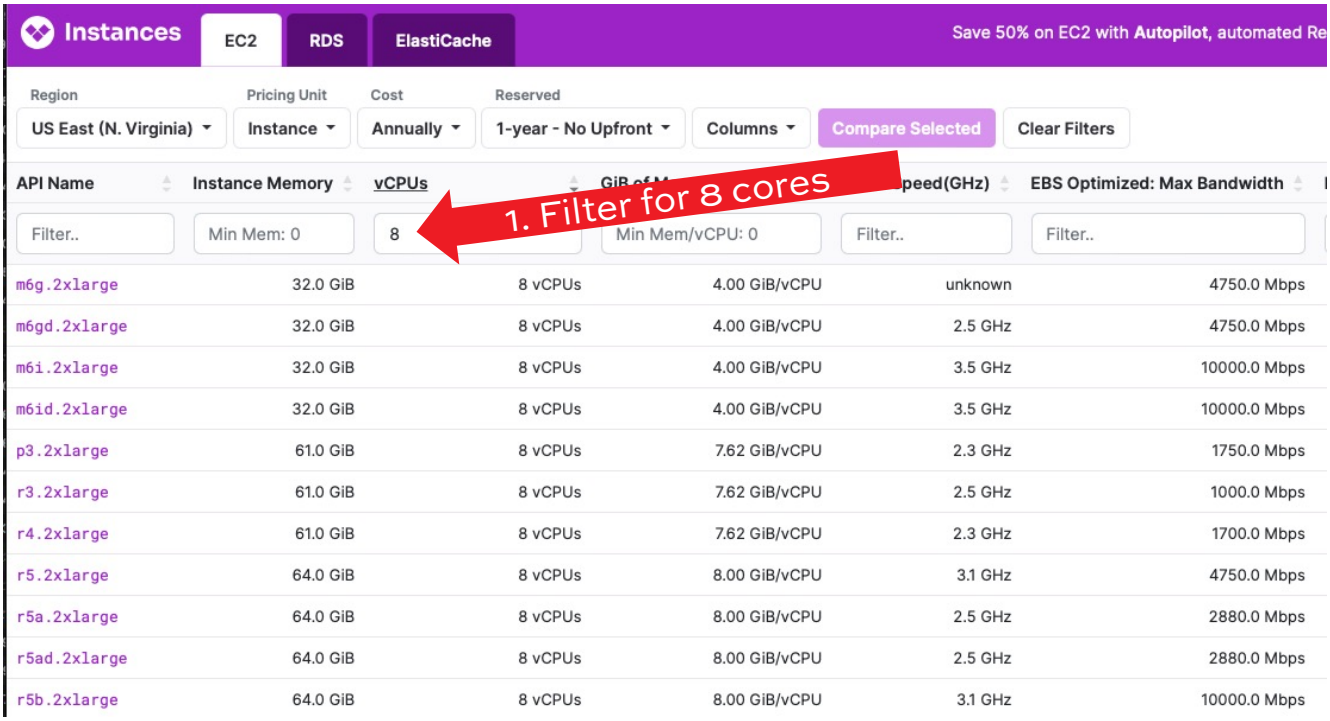


Alex the Architect's current SQL Server:

- 8 cores of Standard Edition licensing
- 64GB RAM
- Database is about 250GB (too big to fit in memory)

What hardware do we get in the cloud?

Deck 4 p67




| API Name | Instance Memory | vCPUs | GiB of M | peed(GHz) | EBS Optimized: Max Bandwidth |
|---|---|---|---|---|---|
| Filter.. | Min Mem: 0 | 8 | Min Mem/vCPU: 0 | Filter.. | Filter.. |
| m6g.2xlarge | 32.0 GiB | 8 vCPUs | 4.00 GiB/vCPU | unknown | 4750.0 Mbps |
| m6gd.2xlarge | 32.0 GiB | 8 vCPUs | 4.00 GiB/vCPU | 2.5 GHz | 4750.0 Mbps |
| m6i.2xlarge | 32.0 GiB | 8 vCPUs | 4.00 GiB/vCPU | 3.5 GHz | 10000.0 Mbps |
| m6id.2xlarge | 32.0 GiB | 8 vCPUs | 4.00 GiB/vCPU | 3.5 GHz | 10000.0 Mbps |
| p3.2xlarge | 61.0 GiB | 8 vCPUs | 7.62 GiB/vCPU | 2.3 GHz | 1750.0 Mbps |
| r3.2xlarge | 61.0 GiB | 8 vCPUs | 7.62 GiB/vCPU | 2.5 GHz | 1000.0 Mbps |
| r4.2xlarge | 61.0 GiB | 8 vCPUs | 7.62 GiB/vCPU | 2.3 GHz | 1700.0 Mbps |
| r5.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 4750.0 Mbps |
| r5a.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 2.5 GHz | 2880.0 Mbps |
| r5ad.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 2.5 GHz | 2880.0 Mbps |
| r5b.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 10000.0 Mbps |

© Brent Ozar Unlimited. All rights reserved.

Instances | EC2 | RDS | ElastiCache | Save 50% on EC2 with **Autopilot**, automated Re

Region: US East (N. Virginia) | Pricing Unit: Instance | Cost: Annually | Reserved: 1-year - No Upfront | Columns | Compare Selected | Clear Filters

| API Name | Instance Memory | vCPUs | GiB of Memory per vCPU | Clock Speed(GHz) | EBS Optimized: Max Bandwidth |
|---|---|---|---|---|---|
| Filter.. | Min Mem: 0 | 8 | Min Mem/vCPU: 0 | Filter.. | Filter.. |
| m6g.2xlarge | 32.0 GiB | 8 vCPUs | [illegible] GiB/vCPU | unknown | 4750.0 Mbps |
| m6gd.2xlarge | 32.0 GiB | [illegible] | 4.00 GiB/vCPU | 2.5 GHz | 4750.0 Mbps |
| m6i.2xlarge | 32.0 GiB | 8 vCPUs | 4.00 GiB/vCPU | 3.5 GHz | 10000.0 Mbps |
| m6id.2xlarge | 32.0 GiB | 8 vCPUs | [illegible] GiB/vCPU | [illegible] | 10000.0 Mbps |
| p3.2xlarge | 61.0 GiB | 8 vCPUs | 7.62 GiB/vCPU | [illegible] GHz | 1750.0 Mbps |
| r3.2xlarge | 61.0 GiB | 8 vCPUs | 7.62 GiB/vCPU | 2.5 GHz | 1000.0 Mbps |
| r4.2xlarge | 61.0 GiB | 8 vCPUs | 7.62 GiB/vCPU | 2.3 GHz | 1700.0 Mbps |
| r5.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 4750.0 Mbps |
| r5a.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 2.5 GHz | 2880.0 Mbps |
| r5ad.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 2.5 GHz | 2880.0 Mbps |
| r5b.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 10000.0 Mbps |

Some have low RAM...

but fast CPUs

and fast IO pipes.

Instances | EC2 | RDS | ElastiCache | Save 50% on EC2 with **Autopilot**, automated Re

Region: US East (N. Virginia) | Pricing Unit: Instance | Cost: Annually | Reserved: 1-year - No Upfront | Columns | Compare Selected | Clear Filters

| API Name | Instance Memory | vCPUs | GiB of Memory per vCPU | Clock Speed(GHz) | EBS Optimized: Max Bandwidth |
|---|---|---|---|---|---|
| Filter.. | Min Mem: 0 | 8 | Min Mem/vCPU: 0 | Filter.. | Filter.. |
| m6g.2xlarge | 32.0 GiB | 8 vCPUs | 4.00 GiB/vCPU | unknown | 4750.0 Mbps |
| m6gd.2xlarge | 32.0 GiB | 8 vCPUs | 4.00 GiB/vCPU | 2.5 GHz | 4750.0 Mbps |
| m6i.2xlarge | 32.0 GiB | 8 vCPUs | 4.00 GiB/vCPU | 3.5 GHz | 10000.0 Mbps |
| m6id.2xlarge | 32.0 GiB | 8 vCPUs | 4.00 GiB/vCPU | 3.5 GHz | 10000.0 Mbps |
| p3.2xlarge | 61.0 GiB | [illegible] | [illegible] | 2.3 GHz | 1750.0 Mbps |
| r3.2xlarge | 61.0 GiB | 8 vCPUs | 7.62 GiB/vCPU | 2.5 GHz | 1000.0 Mbps |
| r4.2xlarge | 61.0 GiB | 8 vCPUs | [illegible] | 2.3 GHz | 1700.0 Mbps |
| r5.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | [illegible] | 4750.0 Mbps |
| r5a.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 2.5 GHz | 2880.0 Mbps |
| r5ad.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 2.5 GHz | 2880.0 Mbps |
| r5b.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 10000.0 Mbps |

Some have lots of RAM

but slow CPUs

and slow IO pipes.

© Brent Ozar Unlimited. All rights reserved.

Instances | EC2 | RDS | ElastiCache | Save 50% on EC2 with **Autopilot**, automated Re

Region: US East (N. Virginia) | Pricing Unit: Instance | Cost: Annually | Reserved: 1-year - No Upfront | Columns | Compare Selected | Clear Filters

| API Name | Instance Memory | vCPUs | GiB of Memory per vCPU | Clock Speed(GHz) | EBS Optimized: Max Bandwidth |
|---|---|---|---|---|---|
| Filter.. | Min Mem: 0 | 8 | Min Mem/vCPU: 0 | Filter.. | Filter.. |
| m6g.2xlarge | 32.0 GiB | 8 vCPUs | 4.00 GiB/vCPU | unknown | 4750.0 Mbps |
| m6gd.2xlarge | 32.0 GiB | 8 vCPUs | 4.00 GiB/vCPU | 2.5 GHz | 4750.0 Mbps |
| m6i.2xlarge | 32.0 GiB | 8 vCPUs | 4.00 GiB/vCPU | 3.5 GHz | 10000.0 Mbps |
| m6id.2xlarge | 32.0 GiB | 8 vCPUs | 4.00 GiB/vCPU | 3.5 GHz | 10000.0 Mbps |
| p3.2xlarge | 61.0 GiB | 8 vCPUs | 7.62 GiB/vCPU | 2.3 GHz | 1750.0 Mbps |
| r3.2xlarge | 61.0 GiB | 8 vCPUs | 7.62 GiB/vCPU | 2.5 GHz | 1000.0 Mbps |
| r4.2xlarge | 61.0 GiB | 8 vCPUs | 7.62 GiB/vCPU | 2.3 GHz | 1700.0 Mbps |
| r5.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 4750.0 Mbps |
| r5a.2xlarge | 64.0 GiB | 8 vCPUs | [illegible] GiB/vCPU | 2.5 GHz | 2880.0 Mbps |
| r5ad.2xlarge | 64.0 GiB | [illegible] | 8.00 GiB/vCPU | [illegible] | 2880.0 Mbps |
| r5b.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 10000.0 Mbps |

Some have lots of RAM

and fast IO, too!

# "Give me fast everything"

**Sure, as you set your thresholds, the options thin out.**

Instances | EC2 | RDS | ElastiCache | Save 50% on EC2 with **Autopilot**, automated Reserved Instance purchasing. | Slack | Contribute

Region: US East (N. Virginia) | Pricing Unit: Instance | Cost: Hourly | Reserved: 1-year - No Upfront | Columns | Compare Selected | Clear Filters | Export | Search...

| API Name | Instance Memory | vCPUs | GiB of Memory per vCPU | Clock Speed(GHz) | EBS Optimized: Max Bandwidth | Reserved cost |
|---|---|---|---|---|---|---|
| Filter.. | 64 | 8 | Min Mem/vCPU: 3. | | Filter.. | Filter.. |
| x2iedn.2xlarge | 256.0 GiB | 8 vCPUs | 32.00 GiB/vCPU | 3.5 GHz | 20000.0 Mbps | $12210.213600 annually |
| d3.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 2800.0 Mbps | $8733.720000 annually |
| i3en.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 4750.0 Mbps | $8619.840000 annually |
| r5.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 4750.0 Mbps | $6009.360000 annually |
| r5d.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 4750.0 Mbps | $6403.560000 annually |
| i4i.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.5 GHz | 10000.0 Mbps | $7121.880000 annually |
| r6i.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.5 GHz | 10000.0 Mbps | $6144.264000 annually |
| r5dn.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 4750.0 Mbps | $6911.640000 annually |
| r6id.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.5 GHz | 10000.0 Mbps | $6553.356000 annually |
| r5b.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 10000.0 Mbps | $6512.884800 annually |

Period trick

Broken right now

© Brent Ozar Unlimited. All rights reserved.

# Then pick out the good deals.

Sure, as you set your thresholds, the options thin out.

Instances | EC2 | RDS | ElastiCache | Save 50% on EC2 with **Autopilot**, automated Reserved Instance purchasing. | Slack | Contribute

Region: US East (N. Virginia) | Pricing Unit: Instance | Cost: Hourly | Reserved: 1-year - No Upfront | Columns | Compare Selected | Clear Filters | Export | Search...

| API Name | Instance Memory | vCPUs | GiB of Memory per vCPU | Clock Speed(GHz) | EBS Optimized: Max Bandwidth | Windows Reserved cost |
|---|---|---|---|---|---|---|
| Filter.. | 64 | 8 | Min Mem/vCPU: 0 | 3. | Filter.. | Filter.. |
| x2iedn.2xlarge | 256.0 GiB | 8 vCPUs | 32.00 GiB/vCPU | 3.5 GHz | 20000.0 Mbps | $12210.213600 annually |
| d3.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 2800.0 Mbps | $8733.720000 annually |
| i3en.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 4750.0 Mbps | $8619.840000 annually |
| r5.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 4750.0 Mbps | $6009.360000 annually |
| r5d.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 4750.0 Mbps | $6403.560000 annually |
| i4i.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.5 GHz | 10000.0 Mbps | $7121.880000 annually |
| r6i.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.5 GHz | 10000.0 Mbps | $6144.264000 annually |
| r5dn.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 4750.0 Mbps | $6911.640000 annually |
| r6id.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.5 GHz | 10000.0 Mbps | $6553.356000 annually |
| r5b.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 10000.0 Mbps | $6512.884800 annually |

Winner

Fast

Cheap

https://aws.amazon.com/ec2/instance-types/r6i/

aws | Contact Us | Support | English | My Account | Sign In to the Console

Products | Solutions | Pricing | Documentation | Learn | Partner Network | AWS Marketplace | Customer Enablement | Events | Explore More

Amazon EC2 | Overview | Features | Pricing | Instance Types | FAQs | Getting Started | Resources

## Amazon EC2 R6i Instances

Run memory-optimized instances powered by 3rd Generation Intel Xeon Scalable processors

Get started with Amazon EC2 R6i instances

**Innovate Every Application Edition** | Learn how to upgrade your application at our free online conference on 9/28 »

Amazon EC2 R6i instances are powered by 3rd Generation Intel Xeon Scalable processors and deliver up to 15% better price performance compared to R5 instances. R6i instances feature an 8:1 ratio of memory to vCPU, just like R5 instances, and support up to 128 vCPUs per instance, which is 33% more than R5 instances. These instances are SAP-Certified and are an ideal fit for memory-intensive workloads, such as SQL and NoSQL databases; distributed web scale in-memory caches, such as Memcached and Redis; in-memory databases, such as SAP HANA; and real-time big data analytics, such as Apache Hadoop and Apache Spark clusters. R6id instances come with local NVMe-based SSD block-level storage for applications that need high-speed, low-latency local storage. Compared to previous generation R5d instances, R6id instances offer 58% higher TB storage per vCPU and 34% lower cost per TB.

**Amazon EC2 R6id instances now available**

R6id instances are powered by 3rd Generation Intel Xeon Scalable processors with an all-core turbo frequency of 3.5 GHz, up to 7.6 TB of local NVMe-based SSD block-level storage, and up to 15% better price performance than previous generation instances. R6id are ideal for a wide range of memory-intensive workloads including those that need access to high-speed, low-latency local storage.

© Brent Ozar Unlimited. All rights reserved.

## https://aws.amazon.com/ec2/instance-types/r6i/

Amazon EC2 R6i instances are powered by 3rd Generation Intel Xeon Scalable processors and deliver up to 15% better price performance compared to R5 instances. R6i instances feature an 8:1 ratio of memory to vCPU, just like R5 instances, and support up to 128 vCPUs per instance, which is 33% more than R5 instances. These instances are SAP-Certified and are an ideal fit for memory-intensive workloads, such as SQL and NoSQL databases; distributed web scale in-memory caches, such as Memcached and Redis; in-memory databases, such as SAP HANA; and real-time big data analytics, such as Apache Hadoop and Apache Spark clusters. R6id instances come with local NVMe-based SSD block-level storage for applications that need high-speed, low-latency local storage. Compared to previous generation R5d instances, R6id instances offer 58% higher TB storage per vCPU and 34% lower cost per TB.

**Amazon EC2 R6id instances now available**

R6id instances are powered by 3rd Generation Intel Xeon Scalable processors with an all-core turbo frequency of 3.5 GHz, up to 7.6 TB of local NVMe-based SSD block-level storage, and up to 15% better price performance than previous generation instances. R6id are ideal for a wide range of memory-intensive workloads including those that need access to high-speed, low-latency local storage.

Came out November 2021.



## Gradual rollouts by region

When the R6i dropped, wasn't available everywhere:

**Availability and Pricing**

EC2 R6i instances are available today in four AWS Regions: US East (N. Virginia), US West (Oregon), US East (Ohio), Europe (Ireland). As usual with EC2, you pay for what you use. For more information, see the EC2 pricing page.

Then one month later:

Posted On: Dec 13, 2021

Starting today, Amazon EC2 R6i instances are available in additional AWS Regions US West (N. California), Asia Pacific (Mumbai), Asia Pacific (Seoul), Asia Pacific (Singapore), Asia Pacific (Sydney), Asia Pacific (Tokyo), Europe (London), and Europe (Paris). Designed for



© Brent Ozar Unlimited. All rights reserved.

## Why everyone doesn't use the same instance types

Not all instance types are available everywhere

Not everyone can take outages to move to new types

Not everyone needs fast CPUs, lots of RAM, fast storage throughput

Old VM types can be less expensive
(but pricing is all over the map, changes frequently)

Deck 4 p77

## What we're going to need to know

How much memory do we have today?

How much storage throughput do we have today?
(Because if data isn't cached, this is the bottleneck.)

How many cloud drives would it take to get that same throughput?

What VM type do we need to handle that much throughput?

Could we compensate by throwing memory in the box?

Deck 4 p78

© Brent Ozar Unlimited. All rights reserved.

# For example, say…

Our current server: 6,000 MB/sec read throughput.

1 EBS Provisioned IOPs volume max: 1,000 MB/sec

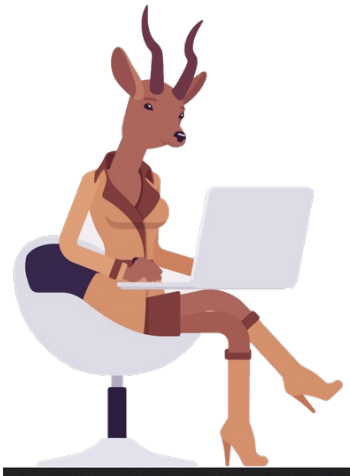
6,000 / 1,000 = we need at least 6 EBS volumes.

And our EC2 virtual machine needs to be able to handle 6,000 MB/sec – many can't.

| API Name | Instance Memory | vCPUs | GiB of Memory per vCPU | Clock Speed(GHz) | EBS Optimized: Max Bandwidth | Windows Reserved cost |
|---|---|---|---|---|---|---|
| Filter.. | 64 | 8 | Min Mem/vCPU: 0 | 3. | Filter.. | Filter.. |
| x2iedn.2xlarge | 256.0 GiB | 8 vCPUs | 32.00 GiB/vCPU | 3.5 GHz | 20000.0 Mbps | $12210.213600 annually |
| d3.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 2800.0 Mbps | $8733.720000 annually |
| i3en.2xlarge | 64.0 GiB | 8 vCPUs | 8.00 GiB/vCPU | 3.1 GHz | 4750.0 Mbps | $8619.840000 annually |

Nope

# We don't have all the answers yet.



Alex the Architect's current SQL Server:

- 8 cores of Standard licensing
- 64GB RAM
- Database is about 250GB (too big to fit in memory)

But we haven't measured Alex's storage throughput – or if it's a problem.

We'll dig into that after backups.



© Brent Ozar Unlimited. All rights reserved.

## What we discussed

Storage hardware is hidden in plan read operators

From 2000s to 2020s,
RAM & storage improved 20X-1000X

Cloud hardware regressed back to the 2000s

To handle it, we have to use 2000s-era techniques:

- Get pools of drives for speed, not 1 for capacity
- Pick your VM instance type based on throughput

Deck 4 p82

© Brent Ozar Unlimited. All rights reserved.

# Links to make analysis easier

https://instances.vantage.sh/ - AWS VMs

https://instances.vantage.sh/rds/ - AWS RDS

https://instances.vantage.sh/azure/ - VMs

Azure SQL DB, Managed Instances, Hyperscale: you're stuck with the ol' documentation



© Brent Ozar Unlimited. All rights reserved.